会社追加ダイアログUI設計書

メタデータ

title: 会社追加ダイアログUI設計書
category: UI設計
version: 1.0.0
last_updated: 2026-01-23
author: Development Team
tags: [provider, corporation, dialog, create]

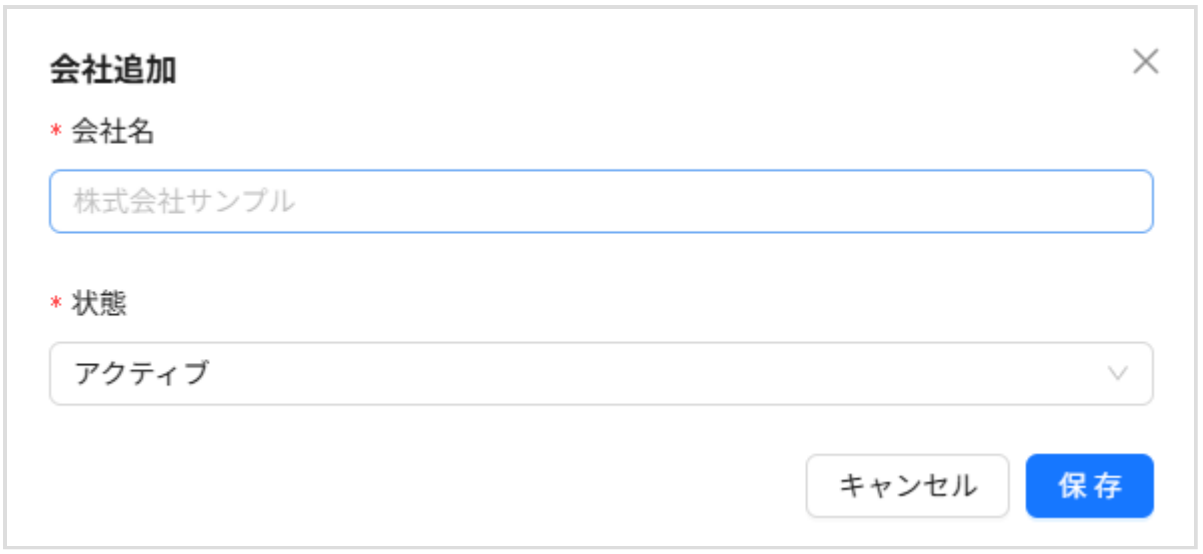
関連ドキュメント

- 会社管理メイン画面
- 会社編集ダイアログ
- マスタ管理API仕様
- 会社テーブルDDL
- 会社管理DB接続計画
- UIポリシー

---

2. 画面レイアウト

2.1. 画面イメージ



---

3. 概要

3.1. 目的

プロバイダが新しい会社を登録するためのダイアログを定義します。会社ID、会社名、アクティブ状態を入力して会社マスタに追加します。

3.2. 利用権限

- **Provider（プロバイダ）**: 使用可能

---

2. ダイアログ構成

2.1. ダイアログ概要

表示トリガー: 会社管理画面の「追加」ボタンクリック

モーダルタイプ: Modal

サイズ: 中（600px）

2.2. レイアウト

```
┌──────────────────────────────────────────┐
│ 会社追加                              × │
├──────────────────────────────────────────┤
│                                          │
│  会社ID *                                │
│  ┌──────────────────────────────────┐    │
│  │                                  │    │
│  └──────────────────────────────────┘    │
│  最大50文字                              │
│                                          │
│  会社名 *                                │
│  ┌──────────────────────────────────┐    │
│  │                                  │    │
│  └──────────────────────────────────┘    │
│  最大50文字                              │
│                                          │
│  状態                                    │
│  ○ アクティブ   ○ 非アクティブ          │
│                                          │
├──────────────────────────────────────────┤
│                    [キャンセル] [保存]   │
└──────────────────────────────────────────┘
```

---

3. 入力項目

3.1. 会社ID（必須）

コンポーネント: `<Form.Item>` + `<Input>`

仕様:

項目	値
ラベル	「会社ID」
プレースホルダー	「会社IDを入力」
型	text
必須	○
最小文字数	1文字
最大文字数	50文字

許可文字	英数字、アンダースコア、ハイフン
重複チェック	有効

バリデーション:

```
{
  required: true,
  message: '会社IDを入力してください'
},
{
  min: 1,
  max: 50,
  message: '会社IDは1〜50文字で入力してください'
},
{
  pattern: /^[a-zA-Z0-9_-]+$/,
  message: '会社IDは英数字、アンダースコア、ハイフンのみ使用できます'
}
```

リアルタイム重複チェック:

- 入力後500msのデバウンス
- 既存の会社IDと重複する場合はエラー表示
- 背景色を黄色（`colorWarningBg`）に変更

3.2. 会社名（必須）

コンポーネント: `<Form.Item>` + `<Input>`

仕様:

項目	値
ラベル	「会社名」
プレースホルダー	「会社名を入力」
型	text
必須	○
最小文字数	1文字
最大文字数	50文字

バリデーション:

```
{
  required: true,
  message: '会社名を入力してください'
},
{
  min: 1,
  max: 50,
  message: '会社名は1〜50文字で入力してください'
}
```

3.3. 状態（必須）

コンポーネント: `<Form.Item>` + `<Radio.Group>`

仕様:

項目	値
ラベル	「状態」
選択肢	アクティブ / 非アクティブ
デフォルト	アクティブ
必須	○

実装例:

```
<Form.Item
  label="状態"
  name="isActive"
  initialValue={true}
  rules={[{ required: true, message: '状態を選択してください' }]}
>
  <Radio.Group>
    <Radio value={true}>アクティブ</Radio>
    <Radio value={false}>非アクティブ</Radio>
  </Radio.Group>
</Form.Item>
```

---

4. アクションボタン

4.1. キャンセルボタン

コンポーネント: `<Button>`

仕様:

項目	値
ラベル	「キャンセル」
タイプ	`default`
配置	フッター左側

動作:

1. 入力内容を破棄
2. ダイアログを閉じる
3. 親画面に戻る

4.2. 保存ボタン

コンポーネント: `<Button>`

仕様:

項目	値
ラベル	「保存」
タイプ	`primary`
配置	フッター右側
ローディング	API呼び出し中は `true`
無効化	バリデーションエラー時

動作:

1. フォームバリデーション実行
2. 会社ID重複チェック

3. API呼び出し: `POST /api/v1/provider/corporation`
4. 成功時:
    - 成功メッセージ表示（Toast）
    - ダイアログを閉じる
    - 親画面の一覧を再取得
5. 失敗時: エラーメッセージ表示

---

5. APIリクエスト仕様

5.1. エンドポイント

`POST /api/v1/provider/corporation`

5.2. リクエストヘッダー

```
Content-Type: application/json
Authorization: Bearer {jwt_token}
```

5.3. リクエストボディ

```
{
  "corporationId": "CORP001",
  "corporationName": "テスト会社",
  "isActive": true
}
```

5.4. レスポンス（成功時）

```
{
  "success": true,
  "data": {
    "corporationId": "CORP001",
    "corporationName": "テスト会社",
    "isActive": true,
    "createAt": "2026-01-23T10:30:00.000Z",
    "updateAt": "2026-01-23T10:30:00.000Z"
  },
  "meta": {
    "timestamp": "2026-01-23T10:30:00.000Z",
    "requestId": "req_abc123"
  }
}
```

5.5. レスポンス（エラー時）

```
{
  "success": false,
  "error": {
    "code": "CORPORATION_ID_DUPLICATE",
    "message": "この会社IDは既に使用されています",
    "details": []
  },
  "meta": {
    "timestamp": "2026-01-23T10:30:00.000Z",
    "requestId": "req_abc123"
  }
}
```

---

6. エラーメッセージ

6.1. バリデーションエラー

エラー条件	メッセージ
会社IDが空	「会社IDを入力してください」

会社IDが長すぎる	「会社IDは1〜50文字で入力してください」
会社IDに不正な文字	「会社IDは英数字、アンダースコア、ハイフンのみ使用できます」
会社IDが重複	「この会社IDは既に使用されています」
会社名が空	「会社名を入力してください」
会社名が長すぎる	「会社名は1〜50文字で入力してください」

6.2. APIエラー

エラーコード	メッセージ
`CORPORATION_ID_DUPLICATE`	「この会社IDは既に使用されています」
`VALIDATION_ERROR`	「入力内容に誤りがあります」
`UNAUTHORIZED`	「この操作を実行する権限がありません」
`INTERNAL_SERVER_ERROR`	「サーバーエラーが発生しました。しばらく経ってから再試行してください」

---

7. 実装例（TypeScript + Next.js + Ant Design）

```tsx
// frontend/src/components/Provider/CorporationAddDialog.tsx
'use client';

import React, { useState, useEffect } from 'react';
import { Modal, Form, Input, Radio, Button, message, theme } from 'antd';
import apiClient from '@/utils/apiClient';

const { useToken } = theme;

interface CorporationAddDialogProps {
  open: boolean;
  onClose: () => void;
  onSuccess: () => void;
}

interface CorporationFormValues {
  corporationId: string;
  corporationName: string;
  isActive: boolean;
}
```

```
export default function CorporationAddDialog({
  open,
  onClose,
  onSuccess
}: CorporationAddDialogProps) {
  const { token } = useToken();
  const [form] = Form.useForm();
  const [isLoading, setIsLoading] = useState(false);
  const [isDuplicateChecking, setIsDuplicateChecking] = useState(false);
  const [isDuplicate, setIsDuplicate] = useState(false);

  // 会社ID重複チェック（デバウンス付き）
  useEffect(() => {
    const corporationId = form.getFieldValue('corporationId');
    if (!corporationId || corporationId.length === 0) {
      setIsDuplicate(false);
      return;
    }

    const timer = setTimeout(async () => {
      setIsDuplicateChecking(true);
      try {
        const response = await apiClient.get(
          `/api/v1/provider/corporation/${corporationId}`
        );
        // 既に存在する場合
        if (response.data.success) {
          setIsDuplicate(true);
          form.setFields([
            {
              name: 'corporationId',
              errors: ['この会社IDは既に使用されています']
            }
          ]);
        }
      } catch (error: any) {
        // 404の場合は重複なし
        if (error.response?.status === 404) {
          setIsDuplicate(false);
        }
      } finally {
        setIsDuplicateChecking(false);
      }
    }, 500);

    return () => clearTimeout(timer);
  }, [form.getFieldValue('corporationId'), form]);

  const handleSubmit = async (values: CorporationFormValues) => {
    if (isDuplicate) {
```

```
      message.error('会社IDが重複しています');
      return;
    }

    setIsLoading(true);
    try {
      const response = await apiClient.post(
        '/api/v1/provider/corporation',
        {
          corporationId: values.corporationId,
          corporationName: values.corporationName,
          isActive: values.isActive
        }
      );

      if (response.data.success) {
        message.success('会社を追加しました');
        form.resetFields();
        onSuccess();
        onClose();
      }
    } catch (error: any) {
      const errorMessage =
        error.response?.data?.error?.message || '会社の追加に失敗しました';
      message.error(errorMessage);
    } finally {
      setIsLoading(false);
    }
  };

  const handleCancel = () => {
    form.resetFields();
    setIsDuplicate(false);
    onClose();
  };

  return (
    <Modal
      title="会社追加"
      open={open}
      onCancel={handleCancel}
      footer={null}
      width={600}
      destroyOnClose
    >
      <Form
        form={form}
        layout="vertical"
        onFinish={handleSubmit}
        initialValues={{ isActive: true }}
```

```
      >
        {/* 会社ID */}
        <Form.Item
          label="会社ID"
          name="corporationId"
          rules={[
            { required: true, message: '会社IDを入力してください' },
            { min: 1, max: 50, message: '会社IDは1〜50文字で入力してください' },
            {
              pattern: /^[a-zA-Z0-9_-]+$/,
              message: '会社IDは英数字、アンダースコア、ハイフンのみ使用できます'
            }
          ]}
          extra="最大50文字"
          validateStatus={isDuplicate ? 'error' : undefined}
        >
          <Input
            placeholder="会社IDを入力"
            maxLength={50}
            style={{
              backgroundColor: isDuplicate ? token.colorWarningBg : undefined
            }}
            suffix={isDuplicateChecking ? <span>確認中...</span> : null}
          />
        </Form.Item>

        {/* 会社名 */}
        <Form.Item
          label="会社名"
          name="corporationName"
          rules={[
            { required: true, message: '会社名を入力してください' },
            { min: 1, max: 50, message: '会社名は1〜50文字で入力してください' }
          ]}
          extra="最大50文字"
        >
          <Input placeholder="会社名を入力" maxLength={50} />
        </Form.Item>

        {/* 状態 */}
        <Form.Item
          label="状態"
          name="isActive"
          rules={[{ required: true, message: '状態を選択してください' }]}
        >
          <Radio.Group>
            <Radio value={true}>アクティブ</Radio>
            <Radio value={false}>非アクティブ</Radio>
          </Radio.Group>
        </Form.Item>
```

```
        {/* フッター */}
        <Form.Item style={{ marginBottom: 0, marginTop: 24 }}>
          <div style={{ display: 'flex', justifyContent: 'flex-end', gap: 8 }}>
            <Button onClick={handleCancel}>キャンセル</Button>
            <Button
              type="primary"
              htmlType="submit"
              loading={isLoading}
              disabled={isDuplicate || isDuplicateChecking}
            >
              保存
            </Button>
          </div>
        </Form.Item>
      </Form>
    </Modal>
  );
}
```

---

8. テスト要件

8.1. 単体テスト

テストケース

- バリデーション機能のテスト
- 重複チェック機能のテスト
- フォーム送信処理のテスト

8.2. 統合テスト

テストシナリオ

1. 正常な会社追加
2. 会社ID重複エラー
3. 必須項目未入力エラー
4. 最大文字数超過エラー

5. 不正な文字列入力エラー

---

9. アクセシビリティ

9.1. キーボード操作

- Tab: 次の入力項目へフォーカス移動
- Shift + Tab: 前の入力項目へフォーカス移動
- Enter: フォーム送信
- Escape: ダイアログを閉じる

9.2. スクリーンリーダー対応

- 各入力項目に適切なラベルを設定
- エラーメッセージを `role="alert"` で通知
- 必須項目を `aria-required="true"` で明示

---

10. 変更履歴

バージョン	日付	変更内容	担当者
1.0.0	2026-01-23	初版作成	Development Team